

Lab 2C: Advanced Skill Workflows - Test, Benchmark & Optimize

Duration: 30 min

After: Advanced Skill Systems lecture (Day 2, Block 3)

Prerequisites: Labs 2A-2B complete

Objective

Run a full **skill eval loop** (test set -> measure -> optimize -> retest), measure **run-to-run variance** so you know how many runs a number needs before you trust it, and run an **A/B check** of your skill against a skills-disabled Claude baseline - then decide: keep, optimize, or retire.

Deliverable (toolkit chain 7 of 12)

A portable eval definition at `.claude/skills/dashboard-code-review/evals/evals.json`, an evidence record at `evals/results.md`, a recorded **variance band**, and a documented A/B verdict - the quality gate for your toolkit's skill.

START MARKER: `/clear done in business-dashboard.`

Fell behind? Run:

Set up this project with: (1) a dashboard-code-review skill at `.claude/skills/dashboard-code-review/SKILL.md` (frontmatter: name, description with review triggers, disable-model-invocation: false, allowed-tools: Read, Grep, Glob, disallowed-tools: Write, Edit, Bash) producing RED/YELLOW/BLUE output, (2) a PostToolUse prettier hook and PreToolUse secret blocker registered in `.claude/settings.json`, and (3) `CLAUDE.md` with project rules. Make scripts executable.

Exercise 1: Build the Eval Test Set (6 min)

Good skills trigger reliably. Prove it with data, not vibes.

```
Create .claude/skills/dashboard-code-review/evals/evals.json as valid JSON with:
- skill_name: "dashboard-code-review"
- an evals array containing 10 objects
- each object has: id, prompt, should_trigger, expected_output, and files

Use these five SHOULD TRIGGER prompts:
1. "Review the code in src/routes/tasks.js"
2. "Find bugs in @src/routes/tasks.js"
3. "QA check on the task routes"
4. "Are there security issues in server.js?"
5. "Check this code for problems"

Use these five SHOULD NOT TRIGGER prompts:
6. "Write a new endpoint for user profiles"
7. "Explain how the Express router works"
8. "Run the tests"
9. "Refactor the dashboard for performance"
10. "Generate API documentation for tasks.js"

For positive cases, expected_output must require dashboard-code-review activation plus the RED/YELLOW/BUE contract and file evidence. For negative cases, expected_output must say the skill remains inactive. Use the relevant project path in files, or an empty array when no file is required.
```

Open `evals/evals.json` in VS Code. Confirm it parses as JSON and contains exactly 10 cases: five true and five false `should_trigger` values.

***Production scale-up:** this classroom set uses 5+5 cases. Agent Skills recommends approximately 8-10 should-trigger and 8-10 should-not-trigger queries for description optimization, with held-out wording reserved for validation.*

Exercise 2: Run the Eval (6 min)

After saving the skill, confirm the existing watched `.claude/skills/` directory live-loads the change, then run `/clear` before each test. If the project's top-level skills directory was created after this session started, restart the Claude Code extension session first. Test at least 3 positives and 2 negatives. Score each:

Prompt	Fired? (RED/YELLOW/BUE seen)	Pass ?
Review the code in src/routes/tasks.js	___	___
Find bugs in @src/routes/tasks.js	___	___
QA check on the task routes	___	___
Write a new endpoint for user profiles	___ (should be NO)	___
Refactor the dashboard for performance	___ (should be NO)	___

Expected output marker: a score like 4/5. Perfect is rare - activation is probabilistic.

Create `.claude/skills/dashboard-code-review/evals/results.md` with this prompt. Replace each [Y/N] with what you observed in the five tests:

Create `.claude/skills/dashboard-code-review/evals/results.md` for the dashboard-code-review activation eval.

Record these five results:

1. Review the code in `src/routes/tasks.js` - should trigger - fired: [Y/N]
2. Find bugs in `@src/routes/tasks.js` - should trigger - fired: [Y/N]
3. QA check on the task routes - should trigger - fired: [Y/N]
4. Write a new endpoint for user profiles - should not trigger - fired: [Y/N]
5. Refactor the dashboard for performance - should not trigger - fired: [Y/N]

For each result, show the prompt, expected behavior, whether it fired, and PASS or FAIL. A positive passes when it fires; a negative passes when it does not fire. Calculate and record the total pass count as X/5. Keep this evaluation limited to the single model-invoked dashboard-code-review skill.

Expected output marker: `evals/results.md` contains five rows and a correctly calculated total such as

4/5. Check the file rather than accepting the summary alone.

Exercise 3: Run Variance - Trust the Band, Not the Run (6 min)

That 4/5 you just wrote down - is it real? One run is an anecdote. For a consistent classroom test, use this exact SHOULD-TRIGGER prompt from Exercise 2:

```
Find bugs in @src/routes/tasks.js
```

This is the weakest positive because it asks for bug finding but does not explicitly say "review" or name the dashboard. Use it even if it failed to trigger in Exercise 2; that uncertainty is what the variance test measures. Do not try to decide that a prompt "barely fired" from one binary result.

Run the exact prompt **5 times**, with `/clear` between runs:

Run	Fired? (Y/N)
1	—
2	—
3	—
4	—
5	—

Record the band: `fired X/5`. Now write it into `evals/results.md`:

```
Append a "Variance" section to .claude/skills/dashboard-code-review/evals/results.md
recording: the prompt tested, the 5-run result (X/5), and this rule of thumb: "No conclusion
from a single run. Five runs minimum before trusting a trigger rate. A change smaller than
the run-to-run spread is noise, not signal."
```

Expected output marker: most people see 3/5-5/5 on a vague prompt - *the same skill, zero changes*. That spread is your noise floor: any "improvement" smaller than it is a coin flip.

Rule of thumb: 5 runs minimum before you trust an eval number; more if the spread is wide. Apply it to everything in Exercise 4.

Exercise 4: A/B Check - Skill vs Skills-Disabled Baseline (8 min)

Does the skill actually improve output, or has the model caught up? Use Claude Code's documented `--disable-slash-commands` option for the baseline. That option disables all skills, including bundled `/code-review`; simply renaming the project skill would restore the bundled skill and confound the comparison.

Step 1 - baseline (all skills OFF):

Exit the interactive session and run this from `business-dashboard`:

```
claude -p --disable-slash-commands "Review src/routes/tasks.js for issues"
```

Save the output to a scratch file. **Baseline validity marker:** the output must contain no RED/YELLOW/BLEU contract and no skill-activation line. If either appears, the baseline is invalid: copy `src/routes/tasks.js` into a temporary directory outside any project `.claude/skills/` tree and rerun the same command there. If a skill still activates, stop the A/B comparison and record the baseline as invalid.

Step 2 - skill ON:

Start a normal Claude Code session in the same directory, run `/clear`, and use the SAME prompt:

```
Review src/routes/tasks.js for issues
```

Confirm that `dashboard-code-review` activates.

Step 3 - compare:

Criteria	Skills disabled	With dashboard-code-review
Structured RED/YELLOW/BLUE output?		
Used the project checklist?		
Cited project-specific conventions?		
Consistent format on a rerun?		

Verdict - read it against your Exercise 3 band:

Result	Action
Skill wins clearly (gap bigger than your band)	Keep
Skill wins slightly (gap inside your band)	No conclusion - rerun more, or call it a tie
Skills-disabled baseline just as good	Consider retiring - the model caught up

Two skill types: capability uplift (teaches something new - keeps its edge) vs workflow/preference (enforces YOUR format - models catch up). Yours is a workflow skill: its value is consistency, not capability.

Exercise 5: Optimize the Description (3 min)

Feed real failures back into the description:

```
Read my dashboard-code-review skill at .claude/skills/dashboard-code-review/SKILL.md.
Read the test definitions in .claude/skills/dashboard-code-review/evals/evals.json and the
evidence in evals/results.md.
Based on the failed or uncertain cases,
Suggest an improved description: sharper trigger phrases, tighter negative boundaries,
clearer separation from documentation and refactoring tasks. Update only the description -
keep the workflow unchanged.
```

After saving the description, confirm the existing watched skill directory live-loads the change, run `/clear`, and rerun the failed prompts. **Expected output marker:** previously-failing prompts now behave - and if the improvement is smaller than your band, rerun before you celebrate.

Exercise 6: Hygiene Check (1 min)

Run `/doctor`. **Expected output marker:** it reports setup health and the documented skill-listing budget/contributors. It is optional and does not replace the activation eval you just recorded.

Optional step. On some builds this runs a lengthy sweep. If it takes more than a minute, cancel it and move on - the eval you just ran is the real evidence.

Optional Challenge: Turn the Eval into a Release Gate (8-10 min)

Finish the core lab first. Append a `RELEASE GATE` section to `evals/results.md` with these requirements:

- At least 4 of 5 `SHOULD TRIGGER` prompts activate the skill.

- At least 4 of 5 SHOULD NOT TRIGGER prompts stay inactive.
- The implementation prompt must never activate the read-only review workflow.
- `Write`, `Edit`, and `Bash` remain disallowed in the skill frontmatter.
- A previously uncertain prompt must pass at least 2 of 3 fresh runs.

Run enough remaining cases to make an honest `READY` or `NOT READY` decision. Record the counts, the three-run result, any failed case, and the smallest next description change you would test. Do not edit the workflow body to rescue a routing failure.

Challenge done when: the file contains the threshold, evidence, and verdict, and another person could rerun the same gate after a model or description update.

VS Code Acceptance Gate and Process Receipt

Before commit, capture the same supervised workflow used across Day 2:

1. **Orient:** select the target range in the editor and use `Alt+K` or an explicit `@file#line-range`; explain unfamiliar code before editing.
2. **Plan:** review the bounded plan in the Claude Code panel; add an inline comment when the plan misses a constraint.
3. **Approve and implement:** approve only the requested scope and exact permissions.
4. **Inspect diff:** review every changed file in VS Code Source Control and reject unexpected files or authority changes.
5. **Verify:** clear the Problems panel for the touched scope and run the lab's deterministic tests or fixtures in the integrated terminal.
6. **Decide:** record the request, approved plan, changed files, verification evidence, and the human commit/install decision.

A model summary or grader result supports this receipt; it does not replace the diff review.

FINISH MARKER - Success Checklist

- ☐ `evals/evals.json` is valid JSON with 5 positive + 5 negative cases
- ☐ `evals/results.md` records the observed evidence
- ☐ At least 5 prompts scored; trigger rate recorded
- ☐ Variance band recorded from 5 runs of one prompt (X/5 + rule of thumb in the file)
- ☐ A/B run completed with the comparison table filled in
- ☐ Keep/optimize/retire verdict written down - judged against the band, not a single run
- ☐ Description optimized and failed prompts retested
- ☐ Final skill and eval artifacts committed to git
- ☐ (Optional) /doctor skill-listing/setup section reviewed

If Stuck

Problem	Recovery
Baseline still shows a skill workflow	Rerun from a temporary directory outside any project skills tree; if any skill marker remains, record the baseline as invalid and stop the A/B comparison
Skill fires on an implementation prompt	Tighten the negative boundary in the description, then rerun one positive and one negative control
A/B outputs look identical	Your rules match model defaults - add project-specific checks the model can't guess
5 runs won't fit the timebox	Run 3 and note the band is provisional - never run 1
Eval results vary run to run	That IS Exercise 3's lesson - report the band, not the point

Minimum path if behind: complete the 5-positive/5-negative `evals/evals.json`, record one uncertain prompt at least three times in `evals/results.md` and label the band provisional, then perform one skills-disabled/normal-session A/B pair against the same rubric. Description optimization is optional until those measurements exist.

Debrief Questions

1. Why rerun the eval set after every model update?
2. Your five-case score improved from 4/5 to 5/5 after a description tweak - what do you check before believing it?
3. Capability uplift vs workflow skill - which survives model progress, and which is yours?
4. What signal would make you retire a skill rather than optimize it?

Standards References

- <https://agentskills.io/skill-creation/optimizing-descriptions>
- <https://agentskills.io/skill-creation/evaluating-skills>